

Forma A

Sistema de Gestión de Cursos y Matrículas

Informe de Integración de Sistemas

Dominio: Forma A

Grupo: Forma A

Integrantes: [Integrante 1], [Integrante 2], [Integrante 3]

Asignatura: Integración de Sistemas

Fecha: Septiembre 2026

Universidad de Concepción — Facultad de Ingeniería

1. Análisis del problema

AprendeMás opera dos sistemas que evolucionaron por separado. Cupos conoce la oferta de cursos y la capacidad restante; Matrículas guarda estudiantes y sus inscripciones. Sin integración, una matrícula puede exceder la capacidad de una sala.

La necesidad es que una matrícula solo se confirme si Cupos la autoriza. Las restricciones relevantes son: API pública versionada, contrato formal, servicios y bases independientes, autenticación y comportamiento seguro ante una dependencia caída.

Se construyó una solución deliberadamente acotada: no intenta introducir una transacción distribuida. En cambio, la decisión de ocupar se mantiene atómica dentro de Cupos y la API informa con precisión los errores recuperables.

2. Diseño y arquitectura

Matrículas recibe HTTP REST bajo /v1. Mantiene estudiantes y matrículas en SQLite. Para verificar disponibilidad usa un canal gRPC reutilizable contra Cupos, que posee una segunda base SQLite. Redis es infraestructura compartida de cachí e idempotencia, no fuente de verdad académica.

Cliente HTTP → Matrículas (REST) → Cupos (gRPC/protobuf)

Esta estructura reduce el acoplamiento de datos: Matrículas persiste el identificador del curso, pero nunca una copia administrable de sus cupos. Docker Compose declara los tres contenedores y variables de conexión.

3. ADR-001: frontera de servicios (D1)

Contexto: Cupos y Matrículas tienen propietarios y datos con sensibilidad diferente. Alternativas: monolito y base común; procesos separados con base común; dos servicios con base por servicio.

Decisión: se eligieron dos servicios con bases separadas. Cupos posee Curso y sus contadores; Matrículas posee Estudiante y Matrícula. La frontera evita que un cambio al esquema de cupos afecte al historial de matrículas.

Costo aceptado: la operación cruza la red y puede requerir conciliación si falla tras ocupar un cupo. Producción evolucionaría hacia reserva identificada y proceso de compensación, no hacia acceso cruzado a datos.

4. ADR-002: REST y gRPC (D2)

Alternativas: REST/JSON para todo, gRPC para todo, o REST externo con gRPC interno. Se eligió la última opción: REST permite a portal y personal usar verbos, códigos y JSON; gRPC genera stubs tipados y ofrece streaming para el catálogo.

El experimento local midió que el Curso de prueba ocupa 39 bytes en protobuf contra 94 bytes JSON. Esa diferencia apoya usar protobuf en el tramo interno repetido.

Modo de invocación: se utiliza unary-unary síncrono para ConsultarCurso, OcuparCupo y LiberarCupo. El cliente Matrículas necesita el resultado inmediato para continuar su lógica de negocio. ListarCursos usa server streaming porque el catálogo puede crecer.

Costo: hay dos contratos que mantener. Se mitiga con cupos.proto y openapi.yaml versionados y con pruebas automáticas de su estructura.

5. ADR-003: contrato y evolución (D3)

REST usa /v1; cambios incompatibles se publican en /v2 con una ventana de deprecación comunicada. Agregar un atributo opcional es compatible, pero modificar semántica o eliminar uno requerido requiere versión nueva.

En protobuf los campos reciben números estables. Una futura descripción = 5 es compatible, pero reenumerar cupos_disponibles no lo es.

6. ADR-004: resiliencia (D4)

Sin timeout, una dependencia lenta puede consumir los recursos de Matrículas. Con disponibilidad basada solo en cachí se podría sobrepasar la sala. Se eligió fail-fast: deadline gRPC de 2 segundos, 503 para UNAVAILABLE y 504 para DEADLINE_EXCEEDED.

Los errores usan application/problem+json con type, title, status y detail. RFC 9457 estandariza esta representación para que clientes HTTP reciban detalle accionable sin una traza de implementación.

Costo: se rechaza temporalmente una matrícula que podría completar más tarde. El cliente debe reintentar con Idempotency-Key.

7. ADR-005: autenticación Bearer

Alternativas: Basic Auth expone credenciales en cada petición; API Key en query string queda expuesta en URLs y logs; Bearer Token usa un token previamente acordado sin credenciales por petición.

Decisión: se eligió Bearer Token con API Key estática para desarrollo y tokens rotables en producción. Se declara en el contrato OpenAPI como securitySchemes: bearerAuth y se aplica globalmente.

Justificación: estándar RFC 6750, soportado nativamente por OpenAPI 3.0; sin credenciales por petición; migrável a JWT sin cambiar la estructura del header.

8. Experimento: serialización (Competencia 6)

Hipótesis: la representación protobuf del mismo Curso utiliza menos bytes y un tiempo de serialización significativamente menor que su equivalente en JSON.

Método: 10 series de 10.000 serializaciones por formato, midiendo con `time.perf_counter_ns()` y `len()`.

Tabla de resultados:

Formato	Tamaño	Media (ns/op)	Desv. est.
Protobuf	39 B	238	72
JSON compacto	94 B	3.770	273

Resultado: protobuf fue 58,5 % menor por mensaje y más rápido.

9. Consideraciones de contexto (Competencia 2)

Factor elegido: privacidad de datos estudiantiles.

Influencia en el diseño:

- Separa la base de Matrículas de la de Cupos para limitar la exposición de nombres y correos al servicio de Cupos.
- Toda la API REST exige Bearer token para acceder a recursos de estudiantes y matrículas.
- Los mensajes de error no incluyen stack traces, devolviendo application/problem+json genéricos.
- Cupos nunca recibe datos personales; solo recibe identificadores de curso.

Costo: tres contenedores más Redis, comunicación por red entre servicios y gestión de tokens.

10. Ejecución con Docker

Requisitos previos: Docker Desktop instalado y en ejecución.

Levantar el sistema:

```
docker compose up --build
```

Este comando construye y levanta los tres contenedores:

- Cupos: servicio gRPC en puerto 50051
- Matrículas: API REST en puerto 8000
- Redis: cachí e idempotencia en puerto 6379

Verificación: la API está en <http://localhost:8000> y Swagger en </docs>.

Probar resiliencia: ejecutar `docker compose stop cupos` y consultar o crear una matrícula; se espera una respuesta HTTP 503 con `application/problem+json`.

Ejemplo de uso:

```
$headers = @{ Authorization = 'Bearer desarrollo-seguro' } $e = Invoke-RestMethod
http://localhost:8000/v1/estudiantes -Method Post -Headers $headers -ContentType application/json
-Body '{"nombre":"Ana Perez","email":"ana@example.com"}' $m = Invoke-RestMethod
http://localhost:8000/v1/matriculas -Method Post -Headers $headers -ContentType application/json
-Body '{"estudiante_id":"'$(($e.id)\'","curso_id":"'ARQ-101\'"}'
```

Pruebas: cuatro pruebas automatizadas pasan: ciclo de ocupar/liberar cupo, curso inexistente, contrato gRPC con streaming, y validación de OpenAPI con Bearer y `problem+JSON`.

11. Reflexión final

¿Qué harían distinto?

- Implementar un sistema de reserva durable con saga pattern para el fallo entre servicios.
- Añadir observabilidad con métricas (Prometheus) y trazabilidad distribuida (OpenTelemetry).
- Realizar pruebas de integración reales en contenedores Docker.
- Implementar TLS entre servicios y rotación automática de tokens.

¿¿Qué quedó pendiente?

- Conciliación de fallo parcial: OcuparCupo succede pero el INSERT de matrícula falla.
- Política de retención de datos y auditoría de operaciones.
- Escalamiento horizontal: SQLite no es adecuado para múltiples instancias; se requeriría PostgreSQL.
- Rate limiting y health checks avanzados.

La solución evita la sobreconstrucción y cumple con todos los requisitos funcionales y de arquitectura planteados.

12. Estructura del repositorio

```
tarea-1/ |- docker-compose.yml |- matriculas/ |      |- Dockerfile |      |- requirements.txt |      |-  
openapi.yaml |      |      __init__.py |      |      main.py |      |      database.py |      |      grpc_client.py  
|      |      cupos_pb2.py |      |      cupos_pb2_grpc.py |      cupos/ |      Dockerfile  
|      cupos.proto |      server.py |      database.py |      tests/ |      test_cupos.py  
|      test_contracts.py |      docs/ |      adr/ |      ADR-001-frontera-servicios.md  
|      ADR-002-rest-grpc.md |      ADR-003-versionado.md  
|      ADR-004-resiliencia.md |      ADR-005-autenticacion.md  
|      experimento_serializacion.py |      generar_informe.py |      requirements-dev.txt  
|      README.md
```

13. Referencias

- [1] gRPC. Performance Best Practices. <https://grpc.io/docs/guides/performance/>
- [2] Protocol Buffers. Language Guide (proto3). <https://protobuf.dev/programming-guides/proto3/>
- [3] IETF. RFC 9457: Problem Details for HTTP APIs. <https://www.rfc-editor.org/rfc/rfc9457.html>
- [4] Redis. Cache-aside. <https://redis.io/docs/latest/develop/use-cases/cache-aside/>
- [5] OpenAPI Specification. <https://swagger.io/specification/>

Anexos del repositorio: openapi.yaml, cupos.proto, cinco ADR, pruebas y guion de video.